

```
!pip install torch torchvision pillow matplotlib
```

```
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.10.0+cu128)
Requirement already satisfied: torchvision in /usr/local/lib/python3.12/dist-packages (0.25.0+cu128)
Requirement already satisfied: pillow in /usr/local/lib/python3.12/dist-packages (11.3.0)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch) (3.24.3)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch) (4.15.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.1)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2025.3.0)
Requirement already satisfied: cuda-bindings==12.9.4 in /usr/local/lib/python3.12/dist-packages (from torch) (12.9.4)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.93)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.90)
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.90)
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (from torch) (9.10.2.21)
Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.4.1)
Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83 in /usr/local/lib/python3.12/dist-packages (from torch) (11.3.3.83)
Requirement already satisfied: nvidia-curand-cu12==10.3.9.90 in /usr/local/lib/python3.12/dist-packages (from torch) (10.3.9.90)
Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90 in /usr/local/lib/python3.12/dist-packages (from torch) (11.7.3.90)
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch) (0.7.1)
Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2.27.5)
Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in /usr/local/lib/python3.12/dist-packages (from torch) (3.4.5)
Requirement already satisfied: nvidia-nvtx-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.90)
Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.93)
Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.13.1.3)
Requirement already satisfied: triton==3.6.0 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.0)
Requirement already satisfied: cuda-pathfinder~=1.1 in /usr/local/lib/python3.12/dist-packages (from cuda-bindings==12.9.4->torch) (1.1.1)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from torchvision) (2.0.2)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.61.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.4.9)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (26.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch) (1.37.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2->torch) (3.0.3)
```

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision.transforms as transforms
import torchvision.models as models
```

```
from PIL import Image
import matplotlib.pyplot as plt
import numpy as np
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(device)
```

```
cuda
```

```
img_size = 512

transform = transforms.Compose([
    transforms.Resize((img_size,img_size)),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485,0.456,0.406],
        std=[0.229,0.224,0.225]
    )
])
```

```
def load_img(path):
```

```
    img = Image.open(path).convert("RGB")
    img = transform(img).unsqueeze(0)

    return img.to(device)
```

```
def denorm(t):
```

```
    mean = torch.tensor([0.485,0.456,0.406]).view(3,1,1).to(t.device)
    std = torch.tensor([0.229,0.224,0.225]).view(3,1,1).to(t.device)
```

```
t = t * std + mean
return torch.clamp(t,0,1)
```

```
content = load_img("content.jpg")
style = load_img("starry_night.jpg")
```

```
vgg = models.vgg19(pretrained=True).features.to(device).eval()

for p in vgg.parameters():
    p.requires_grad=False
```

```
layers = {
    '0': 'conv1',
    '5': 'conv2',
    '10': 'conv3',
    '19': 'conv4',
    '21': 'content',
    '28': 'conv5'
}

def get_features(x):

    feats = {}

    for name, layer in vgg._modules.items():

        x = layer(x)

        if name in layers:
            feats[layers[name]] = x

    return feats
```

```
def gram(x):

    b,c,h,w = x.size()

    f = x.view(c,h*w)

    G = torch.mm(f,f.t())

    return G/(c*h*w)
```

```
content_f = get_features(content)
style_f = get_features(style)

style_grams = {l:gram(style_f[l]) for l in style_f}
```

```
gen = content.clone().requires_grad_(True).to(device)
```

```
optimizer = optim.LBFGS([gen])

style_w = 1e6
content_w = 1
steps = 300
```

```
run=[0]

while run[0] <= steps:

    def closure():

        gen.data.clamp_(-2,2)

        optimizer.zero_grad()

        g_feat = get_features(gen)

        c_loss = torch.mean(
            (g_feat['content']-content_f['content'])**2
        )

        s_loss = 0

        for l in style_f:
```

```

g = gram(g_feat[1])
s = style_grams[1]

s_loss += torch.mean((g-s)**2)

loss = content_w*c_loss + style_w*s_loss

loss.backward()

run[0]+=1

if run[0] % 50 ==0:
    print("step",run[0],"loss",loss.item())

return loss

optimizer.step(closure)

```

```

step 50 loss 17.732213973999023
step 100 loss 9.748456954956055
step 150 loss 8.806981086730957
step 200 loss 8.42345905303955
step 250 loss 8.219173431396484
step 300 loss 8.084500312805176

```

```

fig,ax = plt.subplots(1,3,figsize=(15,5))

ax[0].imshow(denorm(content.squeeze()).permute(1,2,0).cpu())
ax[0].set_title("Content")
ax[0].axis("off")

ax[1].imshow(denorm(style.squeeze()).permute(1,2,0).cpu())
ax[1].set_title("Style")
ax[1].axis("off")

ax[2].imshow(denorm(gen.detach()).squeeze()).permute(1,2,0).cpu())
ax[2].set_title("Stylized Output")
ax[2].axis("off")

plt.show()

```

Content



Style



Stylized Output



```

content = load_img("content2.jpg")
style = load_img("art_style.jpg")

```

```

content_f = get_features(content)
style_f = get_features(style)

style_grams = {layer: gram(style_f[layer]) for layer in style_f}

```

```

gen = content.clone().requires_grad_(True).to(device)

```

```

optimizer = torch.optim.LBFGS([gen])

run = [0]

while run[0] <= steps:

    def closure():

```

```

gen.data.clamp_(-2,2)

optimizer.zero_grad()

g_feat = get_features(gen)

c_loss = torch.mean(
    (g_feat['content'] - content_f['content'])**2
)

s_loss = 0

for l in style_f:
    g = gram(g_feat[l])
    s = style_grams[l]
    s_loss += torch.mean((g - s)**2)

loss = content_w*c_loss + style_w*s_loss

loss.backward()

run[0]+=1

if run[0] % 50 == 0:
    print("step:", run[0], "loss:", loss.item())

return loss

optimizer.step(closure)

```

```

step: 50 loss: 21.6705265045166
step: 100 loss: 14.386159896850586
step: 150 loss: 11.297721862792969
step: 200 loss: 9.942758560180664
step: 250 loss: 9.323987007141113
step: 300 loss: 9.046066284179688

```

```

fig,ax = plt.subplots(1,3,figsize=(16,6))

ax[0].imshow(denorm(content.squeeze()).permute(1,2,0).cpu())
ax[0].set_title("Content")
ax[0].axis("off")

ax[1].imshow(denorm(style.squeeze()).permute(1,2,0).cpu())
ax[1].set_title("Style")
ax[1].axis("off")

ax[2].imshow(denorm(gen.detach().squeeze()).permute(1,2,0).cpu())
ax[2].set_title("Stylized Output")
ax[2].axis("off")

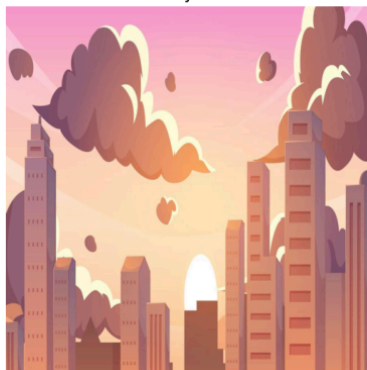
plt.show()

```

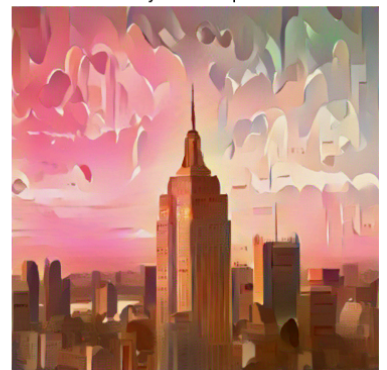
Content



Style



Stylized Output



Start coding or [generate](#) with AI.

